



TECHNISCHE UNIVERSITÄT
CHEMNITZ

Term Paper

Chemnitz Cultural Sites

Subject:

Datenbanken und Webtechniken

Course:

Web Engineering

By:

David Keci

Matriculation number: **858003**

Contents

1	Introduction	2
2	Project Setup	3
2.1	Database overview	3
2.2	Backend part	3
2.3	Frontend part	4
3	User Interface	6
3.1	Login and registration	6
3.2	Page components	7
3.2.1	Home component	7
3.2.2	About Component	8
3.2.3	Map component	9
3.3	Profile Page	12
4.	Conclusion	13
5	References	14
6	Appendix	15

1 Introduction

Chemnitz is a city with a vibrant mix of cultural sites, including museums, theaters, galleries, restaurants, and historical landmarks. However, newcomers, tourists, and even locals often find it difficult to access comprehensive information about these places in one convenient source. The idea for the Chemnitz Cultural Sites web application grew from the recognition that many people want an easier way to discover, explore, and interact with the city’s cultural offerings. In an increasingly digital world, providing a platform that gathers all relevant details—such as location, type, contact information, and visitor experiences—into a single, interactive app can make a real difference for both individuals and the community.

The main goal of this project is to make cultural heritage in Chemnitz more accessible through a modern, user-friendly application. Users can search for cultural sites, filter them by categories like “museum” or “restaurant,” and view results either as a list or directly on an interactive map. Each site features a dedicated page with more detailed information depending on its category. By allowing users to register, create profiles, and manage their list of favorite sites, the application supports a personalized experience. In addition, users can contribute by leaving reviews, making the platform more interactive and community-driven.

Technically, the application is built with the MERN stack—MongoDB, Express.js, React(Next.js), and Node.js—using TypeScript for greater reliability and Tailwind CSS for a clean, responsive interface. This project offered a valuable opportunity to work on full-stack development, combining backend data management, secure authentication, and an intuitive frontend to deliver a comprehensive solution. Overall, the Chemnitz Cultural Sites project aims to promote local culture, support tourism, and help people connect with the city in a more meaningful way.

2 Project Setup

This section explains the technology stack and rationale for each choice.

2.1 Database overview

MongoDB is used as the NoSQL database, chosen for its flexibility and scalability in handling diverse data types and collections.

The main collections:

- **features:** Contains information about cultural sites (theaters, museums, restaurants, galleries, hotels, etc.), each with properties describing its characteristics, address, and geolocation.
- **users:** Stores user data (username, email, hashed password, location, favorites).
- **reviews:** Each review includes a reference to user and feature, a rating, comment, and timestamps.

MongoDB Atlas or a local MongoDB server can be used. During development, **MongoDB Compass** assists with data management.

2.2 Backend part

The backend of the project is implemented using **Node.js**, which provides a robust runtime environment for executing server-side JavaScript. The core framework chosen for backend development is **Express.js**, a minimalist and flexible web framework that allows for the efficient handling of HTTP requests, route definition, and middleware integration.

Database connectivity is managed using the Mongoose library, which offers a schema-based solution for interacting with MongoDB and enables clear definition of models for users, features, and reviews. Upon starting the application, a dedicated connection function initializes the MongoDB database using environment variables, ensuring that database credentials remain secure and configurable.

To facilitate communication between the frontend and backend during development, the application uses the **CORS** (Cross-Origin Resource Sharing) middleware. This is configured to allow requests from the frontend origin and supports the required HTTP methods and credentials, enabling seamless and secure client-server interaction.

Request payloads are parsed using `express.json()`, and cookies are managed through the cookie-parser middleware. These features ensure that the application can correctly interpret incoming data and securely handle user sessions.

The backend follows a **modular architecture**, with separate route files for authentication, user management, and feature (site) management. Each route is mapped to its own controller, promoting code clarity and maintainability. Key routes include:

- **Authentication routes:** Handle user registration, login, logout, and Google OAuth flows.
- **User routes:** Allow users to retrieve and update their profile, manage favorite sites, and add or remove reviews.

- **Feature routes:** Provide endpoints for retrieving all sites, searching by category or name, and accessing individual site details.

Security and user authentication are managed using **JWT (JSON Web Tokens)**, which ensures that protected routes are accessible only to authenticated users. Passwords are securely stored using hashing algorithms.

The backend server runs locally on a configurable port (default: 5000) and logs its status on startup. API endpoints are tested throughout development using Postman.

Overall, the backend is structured to be **secure, scalable, and extensible**, supporting all required functionalities of the application and providing a clean interface for frontend integration.

2.3 Frontend part

The frontend of the project is developed using **Next.js**, a modern React-based framework that supports both server-side rendering and static site generation, enabling optimal performance and improved SEO. TypeScript is employed throughout the frontend codebase to enhance reliability, reduce runtime errors, and ensure maintainability by enforcing static type checking.

Tailwind CSS is used for all styling, providing a utility-first approach that accelerates the design process while guaranteeing a fully responsive interface across devices of all sizes. The result is a clean, modern user interface that remains consistent and intuitive on desktops, tablets, and smartphones.

A key feature of the application is its **interactive map integration**. Utilizing the **Leaflet** library (via the react-leaflet package), the frontend renders a dynamic map on which all cultural sites are displayed as markers. Users can interact with the map by clicking on markers to view detailed site information, and the map automatically updates to reflect applied filters or search results.

In addition to the map, the application presents cultural sites in a visually organized card format. Each card displays relevant information, such as the name, category, address, and rating. Cards are fully interactive—users can save sites to their favorites with a single click or remove them at any time.

Filtering and search functionality are deeply integrated throughout the user experience. Users can filter cultural sites by category with the results updating both on the map and in the list of cards. A **fuzzy search** algorithm is implemented, allowing users to find places even with partial or approximate input, significantly improving accessibility and ease of use.

User-specific features such as favorites are seamlessly managed from the frontend. When a user saves or removes a favorite site, the change is immediately reflected in the user's profile and across all relevant components, ensuring a responsive and interactive experience.

All communication with the backend is performed through secure HTTP requests, with authentication tokens managed to protect user sessions.

Overall, the frontend is designed to be highly interactive, user-friendly, and accessible, making it easy for users to explore, search, and save cultural sites within Chemnitz.

3 User Interface

In this section, the application's main features are presented from the user's perspective, focusing on interaction and usability.

3.1 Login and registration

The application provides a modern and visually appealing authentication interface, supporting both desktop and mobile devices. Users can sign up for a new account or log in to an existing one using a dedicated form. The design utilizes a responsive background image featuring Chemnitz, enhancing the cultural context of the platform.

The **Sign Up** form collects the user's name, email, and password, with the option to register using Google authentication. The **Sign In** form allows users to log in with their email and password or via Google.

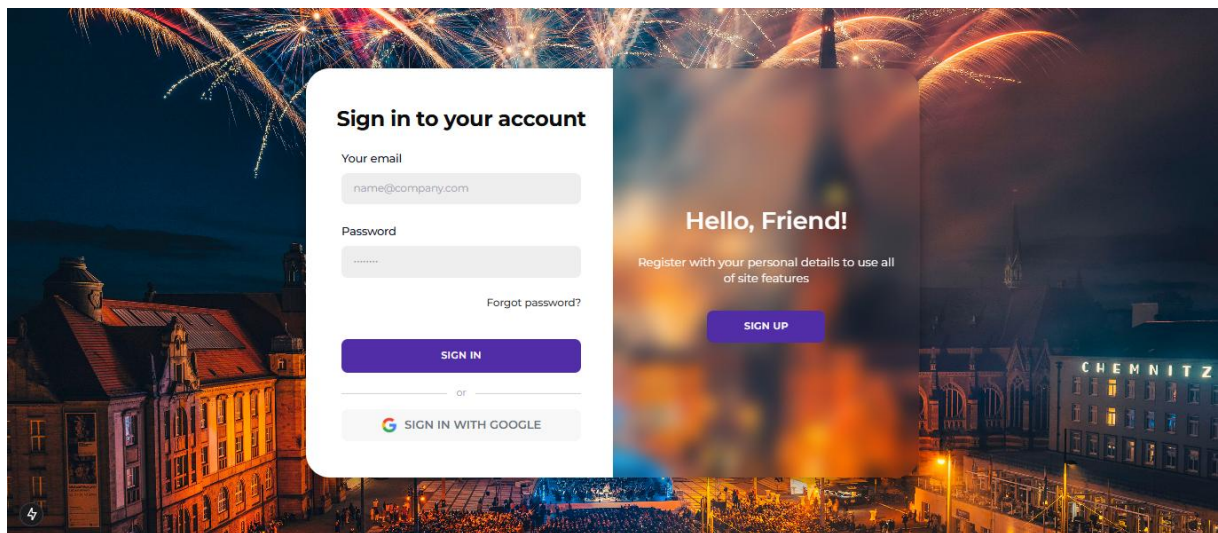


Figure 1: Signin Page

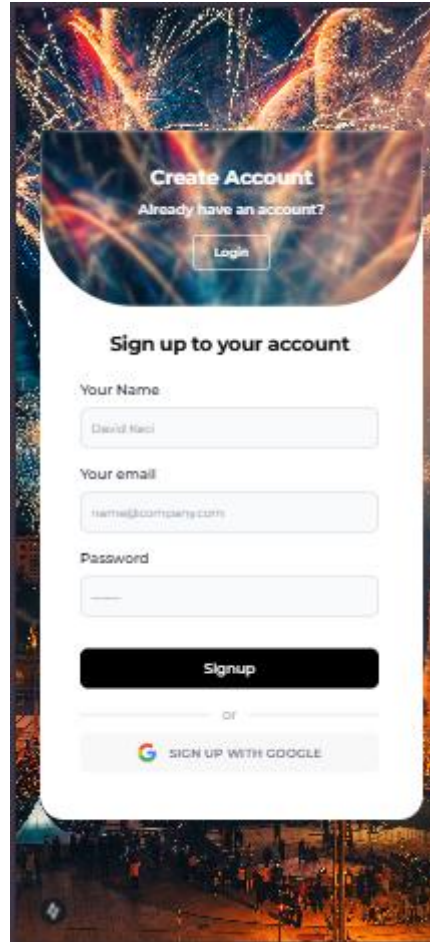


Figure 2: Signin mobile

3.2 Page components

3.2.1 Home component

The main page features a prominent header with navigation links, allowing users to smoothly scroll to different sections such as Home, About, Map, and Contacts. Users can log in or access their profile directly from the header. The page includes a visually engaging carousel that highlights key attractions and places in Chemnitz, providing a dynamic introduction to the city's cultural offerings. The design is fully responsive and ensures intuitive navigation across all devices. (see Figure 3).

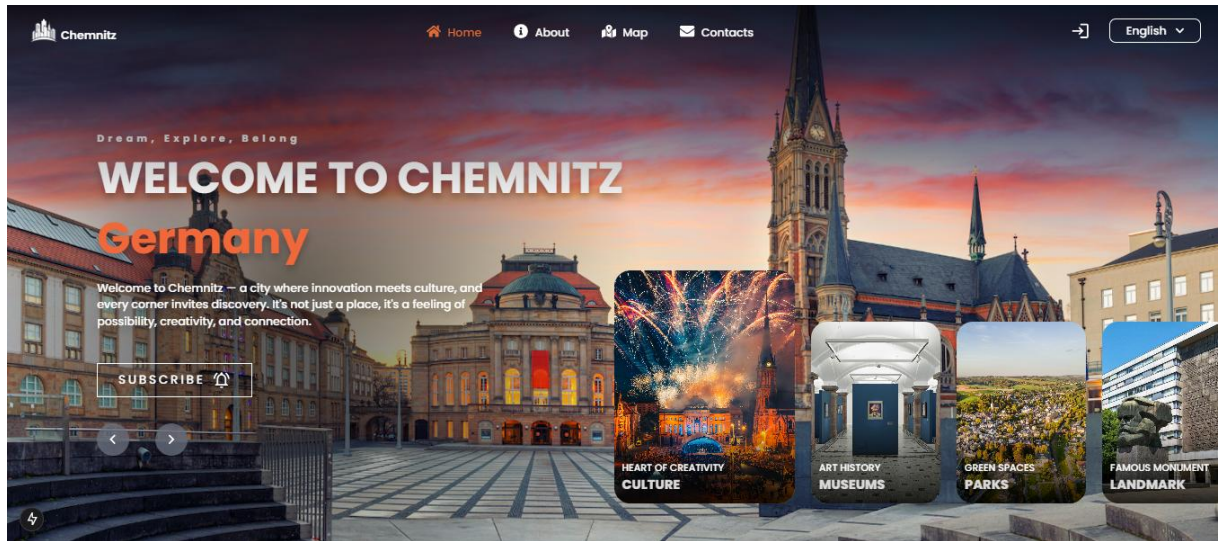


Figure 3: Main Component

3.2.2 About Component

The About component provides users with a visually engaging introduction to the platform and the city. The layout is split into two sections: a prominent headline and descriptive text on the left, and a modern image grid on the right. The design uses large typography and supporting icons to draw attention to key features of the city, while the image grid visually represents different destinations and experiences available to visitors.

Navigation remains consistent with the rest of the site, with the header allowing users to easily access other main sections. The About component is fully responsive, ensuring an attractive and accessible presentation on all devices (see Figure 4).

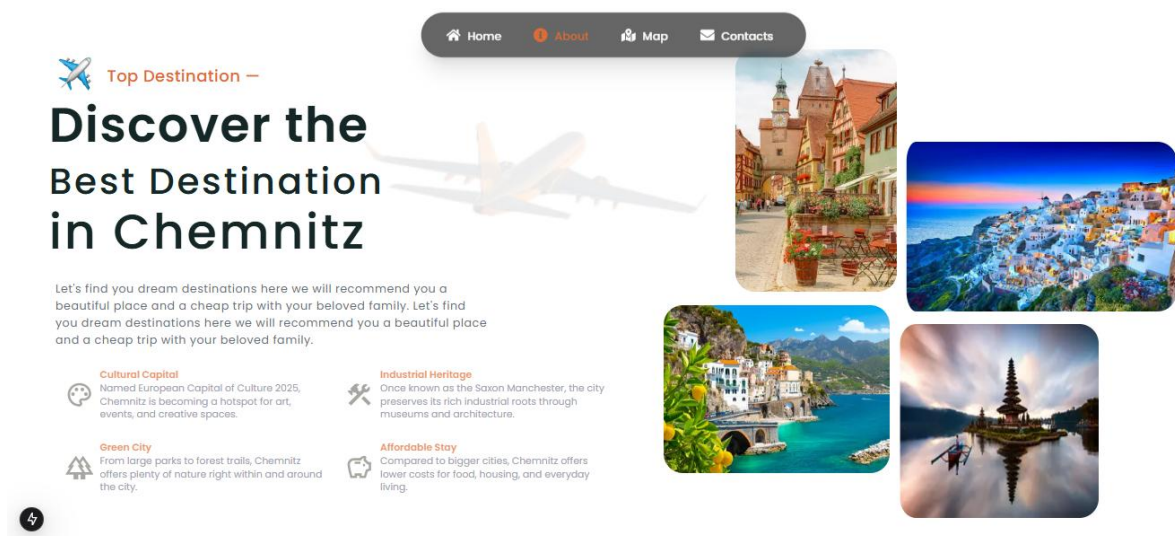


Figure 4: About component

3.2.3 Map component

This core section of the application integrates an interactive map with a card-based listing of cultural sites, enabling users to efficiently discover, filter, and engage with locations across Chemnitz (see Figure 5).

- **Header**

The top of the page features a prominent header that remains consistent throughout the application. Within this header, users find both main navigation links and a dedicated category bar. The navigation menu allows easy access to primary sections such as Home, About, Map, and Contacts. Directly below, a series of category buttons represent different types of destinations—such as museums, galleries, guest houses, hotels, and more. By clicking on a category, users can immediately filter all displayed sites according to their chosen interest, ensuring that only relevant results are shown both on the map and in the card list. Additionally, a “Show Favourite” button enables users to instantly filter the view to only show locations they have previously saved, enhancing personalized exploration.

The header also contains a **search bar**. Here, users can simply begin typing the name or a keyword related to a place; the interface provides instant, fuzzy search suggestions based on partial matches, making it easy to discover specific destinations even with incomplete input.

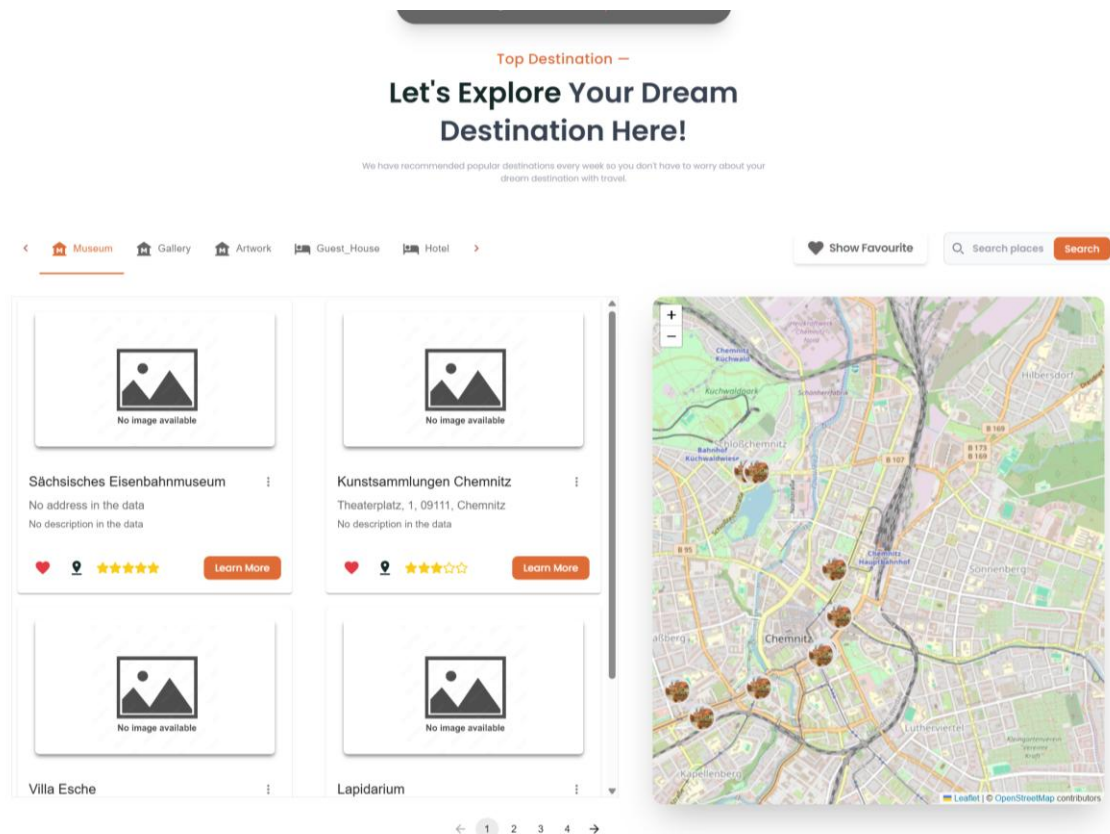


Figure 5: Map Container

- **Cards**

Beneath the header, cultural sites are presented as interactive cards. Each card succinctly summarizes the key details of a location:

- The name of the site and its address (if available),
- A placeholder image representing the location,
- A heart icon for adding or removing the site from the user's favorites,
- A location marker icon for focusing the map on the selected site,
- A star rating showing the site's average user review score,
- And a prominent **"Learn More" button**, which opens a detailed view of the site and provides the option for the user to write and submit a review (see Figure 6).

The card layout allows users to quickly scan through available options.

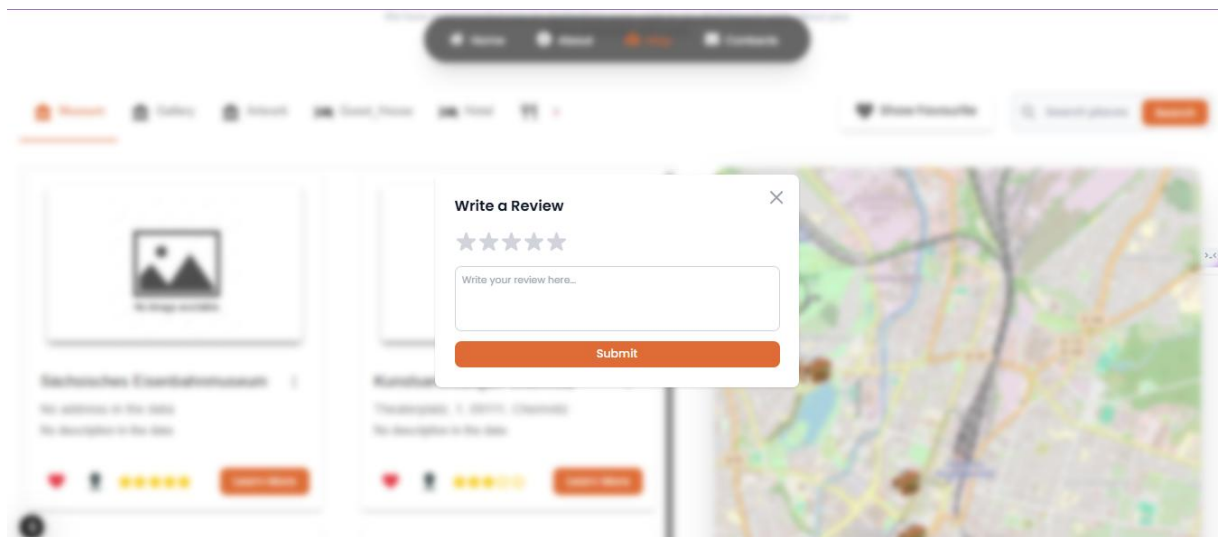
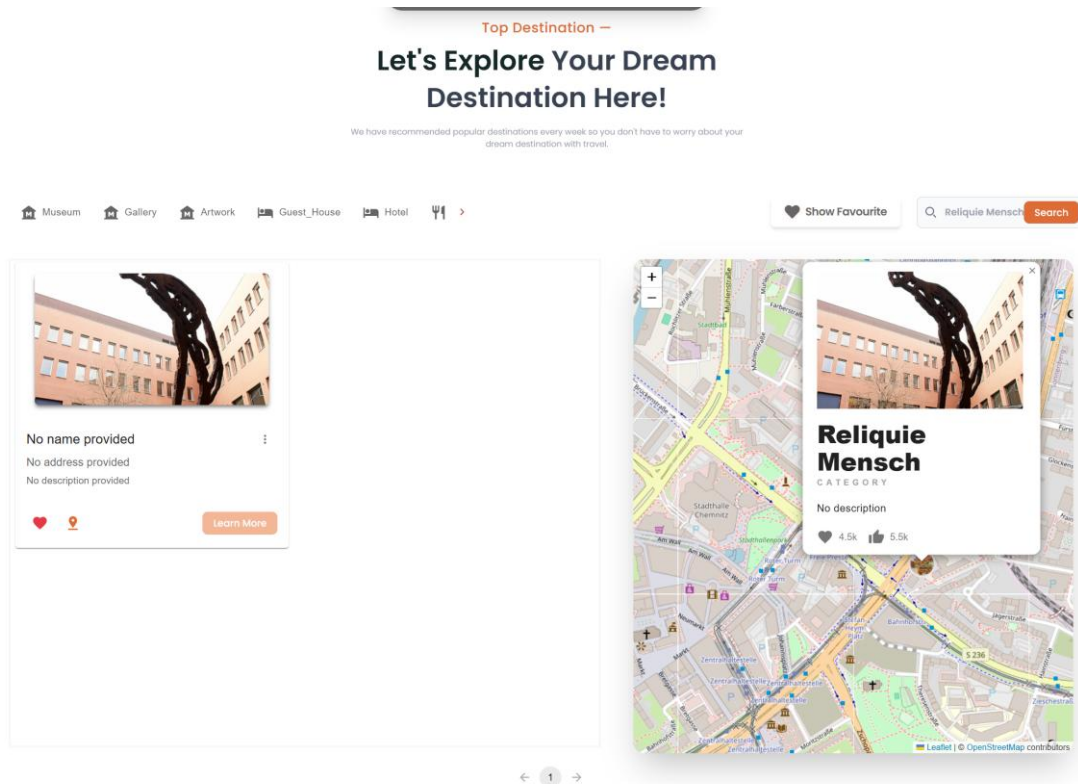


Figure 6: Review

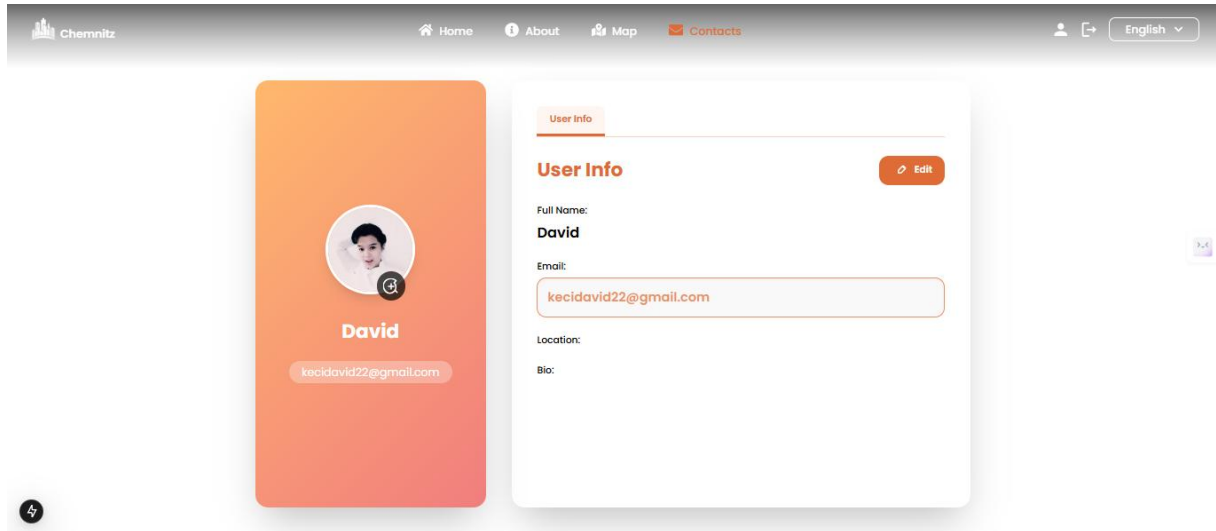
- **Map**

On the right side of the page, the application displays an interactive map using the Leaflet library. This map is fully synchronized with the selected filters and search results—only the relevant sites appear as markers. Users can zoom and pan the map to explore Chemnitz spatially, and clicking on any marker provides instant access to details for that particular site. The map offers a clear, visual overview, complementing the list-based card interface and making it easy for users to orient themselves and plan their visit.



3.3 Profile Page

The Profile page displays the user's personal information, including their name, email, location, and bio. Users can view and update their profile details, as well as change their profile picture, all within a clean and user-friendly interface.



4. Conclusion

This project demonstrates the development of a modern, user-centered web application for exploring cultural sites in Chemnitz. By integrating the MERN stack with technologies such as Next.js, Tailwind CSS, and Leaflet, the application provides a seamless experience that enables users to browse, search, and interact with cultural landmarks both on an interactive map and through detailed cards. Features such as personalized user accounts, favorites management, and user reviews enhance engagement and usability. Overall, the platform successfully meets its goal of making cultural heritage more accessible, interactive, and appealing to both residents and visitors of Chemnitz.

5 References

- MongoDB. (2024). *MongoDB Documentation*. Retrieved from <https://www.mongodb.com/docs/>
- Node.js Foundation. (2024). *Node.js Documentation*. Retrieved from <https://nodejs.org/en/docs/>
- Express.js. (2024). *Express - Node.js web application framework*. Retrieved from <https://expressjs.com/>
- Next.js. (2024). *Next.js Documentation*. Retrieved from <https://nextjs.org/docs>
- React. (2024). *React Documentation*. Retrieved from <https://react.dev/>
- Tailwind Labs. (2024). *Tailwind CSS Documentation*. Retrieved from <https://tailwindcss.com/docs>
- Leaflet. (2024). *Leaflet - a JavaScript library for interactive maps*. Retrieved from <https://leafletjs.com/>
- Mongoose. (2024). *Mongoose ODM Documentation*. Retrieved from <https://mongoosejs.com/docs/>
- Axios. (2024). *Axios Documentation*. Retrieved from <https://axios-http.com/docs/intro>

6 Appendix

Api documentation

- **User Authentication & Profile**
- **POST /api/user/register**

Description:

Register a new user with email, password, and name.

Request Body:

json

CopyEdit

```
{
  "name": "David",
  "email": "kecidavid22@gmail.com",
  "password": "strongPassword123"
}
```

Response (201 Created):

json

CopyEdit

```
{
  "message": "User registered successfully"
}
```

Possible Errors:

- 400: User already exists or invalid input

-
- **POST /api/auth/login**

Description:

Login with email and password, or Google token.

Manual Login Request Body:

json

CopyEdit

```
{
  "email": "kecidavid22@gmail.com",
  "password": "strongPassword123"
}
```

Google Login Request Body:

json

CopyEdit

```
{
  "googleToken": "<GOOGLE_ID_TOKEN>"
}
```

Response:

json

CopyEdit

```
{
```

```
"message": "Login successful"
}
```

Possible Errors:

- 401: Invalid credentials
 - 400: Missing required fields
-

- **POST /api/auth/logout**

Description:

Logs out the authenticated user and clears cookies.

Headers:

- Requires JWT authentication (sent as httpOnly cookie)

Response:

json

CopyEdit

```
{
  "message": "Logout successful"
}
```

- **GET /api/auth/google**

Description:

Redirect to Google OAuth login.

Request Body:

- None (Redirects user to Google login)
-

- **GET /api/auth/google/callback**

Description:

Callback endpoint for Google OAuth login.

Request Body:

- None (Automatically sets cookies and redirects user to the app)
-

- **GET /api/user/getUser**

Description:

Returns current user profile.

Headers:

- Requires JWT authentication

Response:

json

CopyEdit

```
{
  "email": "kecidavid22@gmail.com",
  "name": "David",
  "favourites": ["node/3144070370", "way/23757830"]
}
```

- **PATCH /api/user/profile**

Description:

Update user profile (except email, role, or Google ID).

Request Body (any field to update):

json

CopyEdit

```
{
  "name": "New Name",
  "password": "newPassword123",
  "location": "Chemnitz"
}
```

Response:

json

CopyEdit

```
{
  "email": "kecidavid22@gmail.com",
  "name": "New Name",
  "favourites": []
}
```

-
- **Favourites**
 - **POST /api/user/favourites/add/:featureId**

Description:

Add a place to user's favourites.

URL Parameter:

- featureId (e.g. node/3144070370)

Headers:

- Requires JWT authentication

Response:

json

CopyEdit

```
{
  "favourites": ["node/3144070370", "way/23757830"]
}
```

-
- **POST /api/user/favourites/remove/:featureId**

Description:

Remove a place from favourites.

URL Parameter:

- featureId

Headers:

- Requires JWT authentication

Response:

json

CopyEdit

```
{
  "favourites": ["way/23757830"]
}
```

-
- **GET /api/user/favourites**

Description:

Returns all favourite places for the user.

Headers:

- Requires JWT authentication

Response:

json

CopyEdit

```
[
  {
    "type": "Feature",
    "id": "node/3144070370",
    "properties": { "name": "Emmi am Zöllnerplatz", ... },
    "geometry": { "type": "Point", "coordinates": [12.9271, 50.8463] }
  },
  ...
]
```

- **Features (Cultural Sites)**
- **GET /feature/category/:field/:value**

Description:

Get sites by category.

Example:

GET /feature/category/amenity/restaurant

Response:

json

CopyEdit

```
[
  {
    "id": "node/3144070370",
    "geometry": { "type": "Point", "coordinates": [12.9271, 50.8463] },
    "properties": { "name": "Emmi am Zöllnerplatz", ... },
    "type": "Feature",
    "reviews": [],
    "averageRating": 4.5
  }
]
```

- **GET /feature/id/:id**

Description:

Get details for a specific site by ID.

Example:

GET /feature/id/node/3144070370

Response:

json

CopyEdit

```
{
  "id": "node/3144070370",
```

```
"geometry": { "type": "Point", "coordinates": [12.9271, 50.8463] },
"properties": { "name": "Emmi am Zöllnerplatz", ... },
"type": "Feature",
"reviews": [],
"averageRating": 4.5
}
```

- **GET /feature/fuzzy-name/:name**

Description:

Fuzzy search sites by name (auto-suggestions).

Example:

GET /feature/fuzzy-name/Chemnitz

Response:

json

CopyEdit

```
[
  { "id": "node/3144070370", "name": "Emmi am Zöllnerplatz" },
  { "id": "way/23757830", "name": "Sächsisches Eisenbahnmuseum" }
]
```

- **Reviews**
- **POST /api/user/features/:featureId/review**

Description:

Add a review to a site.

Headers:

- Requires JWT authentication

Request Body:

json

CopyEdit

```
{
  "rating": 5,
  "comment": "Great place for art lovers!"
}
```

Response:

json

CopyEdit

```
{
  "message": "Review added.",
  "reviews": [
    {
      "userEmail": "kecidavid22@gmail.com",
      "rating": 5,
      "comment": "Great place for art lovers!",
      "createdAt": "2025-07-03T19:53:12.123Z"
    }
  ],
  "averageRating": 4.5
}
```

```
}
```

-
- **DELETE** /api/user/features/:featureId/review

Description:

Delete your review for a site.

Headers:

- Requires JWT authentication

Response:

json

CopyEdit

```
{  
  "message": "Review deleted.",  
  "reviews": []  
}
```

-
- **GET** /api/user/features/:featureId/reviews

Description:

Get all reviews for a site.

Response:

json

CopyEdit

```
{  
  "reviews": [  
    {  
      "userEmail": "otheruser@gmail.com",  
      "rating": 4,  
      "comment": "Nice site!",  
      "createdAt": "2025-06-01T09:45:12.123Z"  
    },  
    {  
      "userEmail": "kecidavid22@gmail.com",  
      "rating": 5,  
      "comment": "Great place for art lovers!",  
      "createdAt": "2025-07-03T19:53:12.123Z"  
    }  
  ]  
}
```